

RiskRadar — Tech Stack Reference (Source of Truth)

Version: Post-Meeting Final · March 3, 2026 **Source:** Security Questionnaires (Feb-Mar 2026) — Noah Benoit, Max Compeaux, Rebecca Gautreaux, Celeste George, Qui Huynh, Ben Manuel | Main branch source code | In-person meeting corrections (3/2/26)

Tech Stack Flowchart

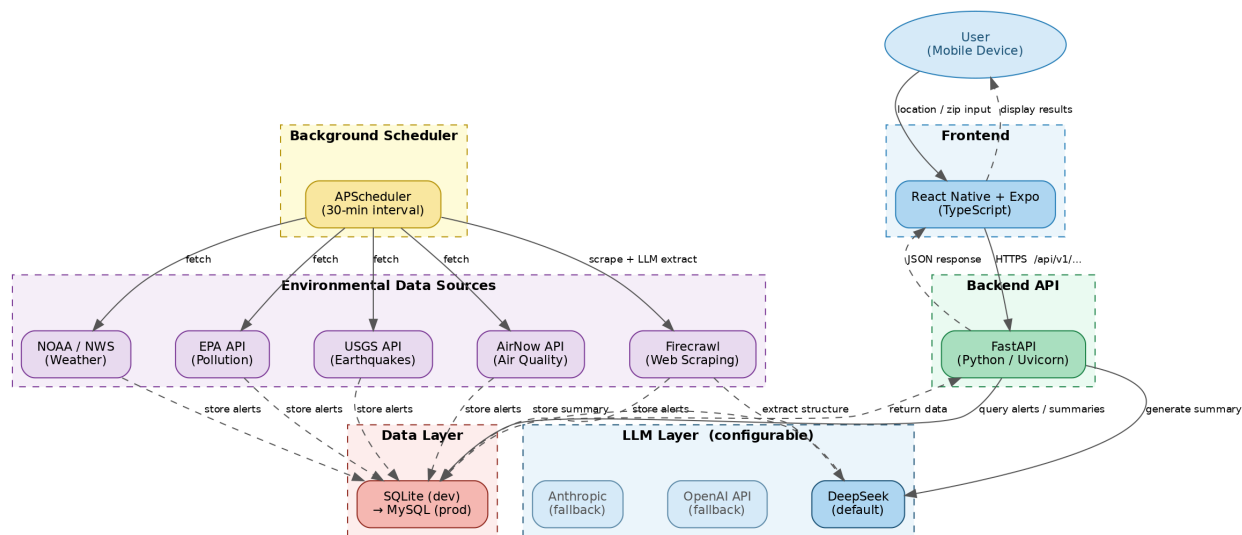


Figure 1: RiskRadar Tech Stack Flowchart

Flowchart Summary: The user's mobile device communicates with a React Native + Expo (TypeScript) frontend, which sends HTTPS requests to a FastAPI backend (Python/Uvicorn). The backend queries the SQLite database (dev) / MySQL (production) for stored alerts and summaries, and calls the LLM layer to generate location-based risk summaries. Separately, APScheduler runs on a 30-minute interval, pulling data from NOAA/NWS (weather), AirNow (air quality), EPA (pollution), USGS (earthquakes), and Firecrawl (web scraping with LLM-assisted extraction). All fetched data is stored back into the database. DeepSeek (deepseek-chat) is the default LLM; OpenAI and Anthropic are configured as fallbacks. NASA FIRMS (wildfire) is currently in the codebase but may be removed.

Confirmed Tech Stack

Platform

Cross-platform mobile application targeting iOS and Android. Delivered via React Native with Expo managed workflow. Web output is also configured (static export) but is not the primary target.

Frontend

Component	Confirmed Value
Framework	React Native 0.81.5
Build toolchain	Expo ~54.0.33 (managed workflow)
Language	TypeScript ~5.9.2
Routing	expo-router ~6.0.23
Navigation	@react-navigation/native, @react-navigation/bottom-tabs
Animations	react-native-reanimated ~4.1.1
Runtime (dev)	Node.js (for React Native Metro bundler)
Testing device	Android SDK (running in VM)
New Architecture	Enabled (newArchEnabled: true)

Backend

Component	Confirmed Value
Framework	FastAPI
Server	Uvicorn (ASGI)
Language	Python
Background scheduler	APScheduler — 30-minute scrape interval
Password hashing	SHA-256 (current) — see security notes
Auth mechanism	JWT (JSON Web Tokens)
Message queue	Not implemented — RabbitMQ under consideration, currently localized

Database

Environment	Database
Development	SQLite
Production (target)	MySQL
ORM	SQLAlchemy (facilitates the SQLite → MySQL migration path)
Schema	13-table MariaDB-compatible schema (see docs/DATA_MODEL.md)

Meeting confirmation (3/2/26): “We are going to be using MySQL instead of SQLite.” SQLite is dev-only and temporary.

LLM Layer

Role	Provider	Model
Default	DeepSeek	deepseek-chat
Fallback	OpenAI	Configurable

Role	Provider	Model
Fallback	Anthropic	Configurable

Meeting note (3/2/26): OpenAI's API costs are a bottleneck. DeepSeek was adopted as the default (LLM_PROVIDER = "deepseek" in settings.py). The LLM provider is configurable via environment settings, with OpenAI and Anthropic available as fallbacks.

Data Sources

Source	Data Type	Status
NOAA / NWS	Weather alerts	Active
AirNow API	Air quality (AQI)	Active
EPA API	Pollution data	Active
USGS API	Earthquakes	Active (added post-meeting)
Firecrawl	Web scraping + LLM extraction	Active — confirmed in-meeting
NASA FIRMS	Wildfire data	In codebase — may be removed

Meeting note (3/2/26): "We may be adding more websites as well as removing the NASA wildfire API." Firecrawl is set in stone as the web scraping solution.

Hosting

AWS (confirmed across multiple questionnaires). Specific services (EC2, Lambda, RDS, etc.) are TBD.

API Overview

Base path: /api/v1/ All endpoints served over HTTPS.

Endpoint	Method	Description
/api/v1/alerts	GET	Retrieve stored alerts (filtered by location/type)
/api/v1/alerts/{id}	GET	Retrieve a single alert by ID
/api/v1/summaries	GET	Retrieve AI-generated summaries
/api/v1/users/register	POST	Create a new user account
/api/v1/users/login	POST	Authenticate and receive JWT

Endpoint	Method	Description
/api/v1/users/preferences	GET/PUT	Retrieve or update alert preferences
/api/v1/users/notifications	GET/PUT	Retrieve or update notification settings
/api/v1/scrape/trigger	POST	Manually trigger a scrape run (admin)
/api/v1/health	GET	Service health check

Authentication

JWT-based authentication. Tokens are issued at login and passed as `Authorization: Bearer <token>` headers on protected routes. Passwords are currently hashed with SHA-256 — see security notes below.

Data Collected

Category	Data Points
Location	ZIP code, latitude, longitude (user-provided at registration)
Account	Display name, email, hashed password
Device	Push notification token, platform (iOS / Android)
Preferences	Alert type preferences (JSON), category preferences, notification settings, quiet hours
Behavior	Articles read, read timestamps, reading progress percentage
Environmental	Weather alerts, AQI readings, pollution data, seismic events, scraped web content
AI Output	Generated summaries, model identifier, token counts

Security Notes

Password Hashing — SHA-256

The current implementation uses SHA-256 for password hashing. SHA-256 is a general-purpose cryptographic hash function — it is fast by design, which makes it unsuitable

for password storage. An attacker with access to the hash database can run high-speed brute-force or dictionary attacks. **Before production deployment, hashing should be migrated to bcrypt or Argon2**, both of which are deliberately slow and include built-in salting. SQLAlchemy makes this a straightforward update to the user model.

CORS — Wildcard Origin

CORS is currently configured as `allow_origins=["*"]` in `main.py`. This permits any domain to make cross-origin requests to the API, which is acceptable for local development but is a security risk in production (enables CSRF-style attacks from arbitrary web origins). **Before deployment, `allow_origins` must be restricted to the specific domains and ports the application will serve from.**

Rate Limiting

No rate limiting is currently implemented on any endpoint. Without rate limiting, the API is vulnerable to brute-force attacks (against `/login`), scraper abuse, and unintentional denial-of-service from misbehaving clients. **Rate limiting should be added before production** — FastAPI supports this cleanly via middleware libraries such as `slowapi`.

Architecture Notes

Scraper Registry

The backend uses a config-driven scraper registry (`backend/config/sources.yaml`) alongside legacy hardcoded scrapers. New API data sources can be added by extending `sources.yaml` without modifying Python code. Firecrawl-based web scrapers are also registered through this config.

RabbitMQ

RabbitMQ appears in the planned architecture doc (`docs/ARCHITECTURE.md`) but is **not implemented** in the current codebase. Background jobs currently run in-process via `APScheduler`. Per meeting (3/2/26): “Need to look into using RabbitMQ, currently localized.”

Database Migration Path

SQLAlchemy ORM abstracts the database layer, making the SQLite → MySQL switch a configuration change (`DATABASE_URL` env variable) rather than a code rewrite. Schema is documented in `docs/DATA_MODEL.md`.

Known Gaps / Open Items

Item	Status	Note
SHA-256 → bcrypt/Argon2	Open	Required before production
CORS wildcard restriction	Open	Required before deployment
Rate limiting	Open	Required before production
RabbitMQ integration	Aspirational	Under consideration, not started
NASA FIRMS removal	Pending decision	May be removed per meeting
Additional Firecrawl web sources	TBD	<code>web_sources: []</code> in <code>sources.yaml</code> is empty but ready
AWS service specifics	TBD	EC2, RDS, Lambda — not yet decided
Push notification delivery	TBD	Device tokens collected; delivery mechanism not confirmed

Document reflects the state of the main branch as of 3/2/26 and the decisions recorded during the in-person team meeting on the same date.